

IITA Rescue-BOT



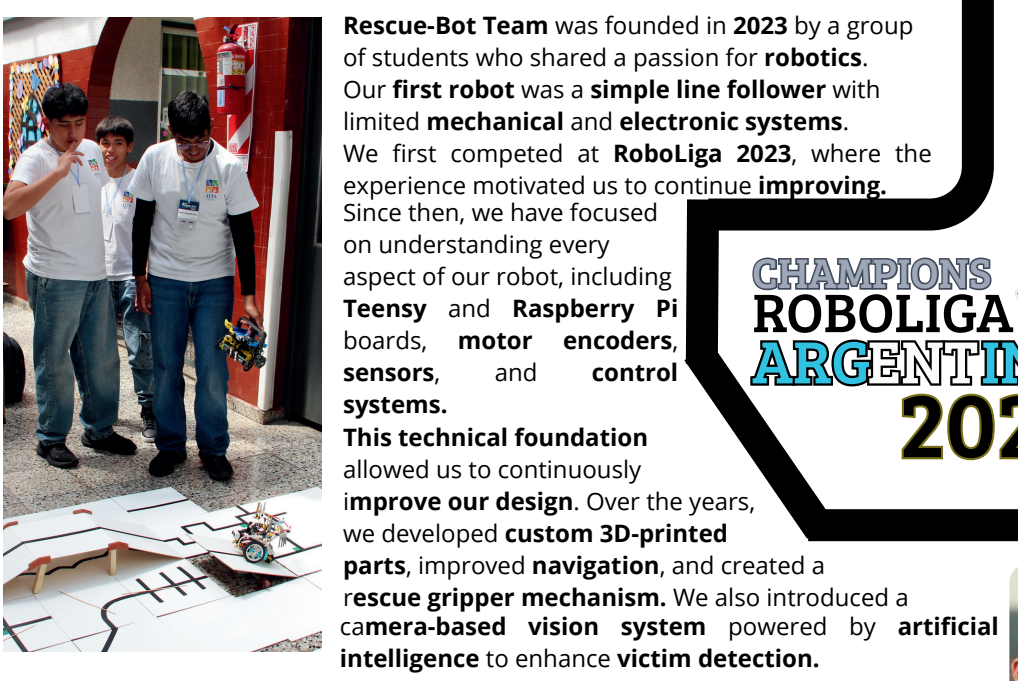
SOFTWARE

TEAM SONG

RoboCupJunior Rescue Line 2026- Salta, Argentina

LINE FOLLOWING

TEAM



Our team played an important role in introducing fully custom electronic robots at our institute, moving beyond LEGO-based platforms and developing robots entirely from scratch. In 2024, we achieved second place at the National Tournament and later won the Innovation Award for learning, development, and teamwork, qualifying for the RoboCup World Championship 2026. Today, our team consists of three members: Benjamin, responsible for electronics integration; Laureano, responsible for 3D design and Teensy programming; and Lucio, responsible for Raspberry Pi and vision system development. Our goal is to continue learning, improving our technology, and sharing experiences with teams from around the world through RoboCup.

Laureano Monteros
I am 16 years old and have been studying robotics since 2019. I started with LEGO robotics and participated in internal competitions, where I developed teamwork, problem-solving, and perseverance skills. In 2023, I transitioned from block programming to text-based programming, learning Python and later C++.

Through robotics challenges, I became particularly interested in low-level control and autonomous systems. Today, I am one of the team's main programmers, focusing on autonomous navigation, sensor integration, robot control, and software development for Rescue Line competitions.

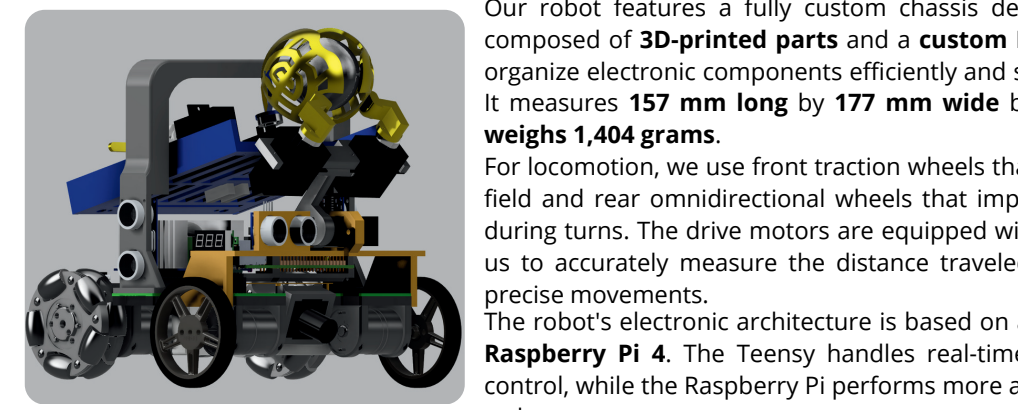
Lucio Saucedo
I am 15 years old and have been involved in robotics since I was 9, starting with LEGO Robotics and learning the fundamentals of programming, robotics, and teamwork. Since 2023, I have focused on text-based programming and autonomous systems, improving my programming and problem-solving skills through robotics projects and competitions.

Within the team, I am responsible for Raspberry Pi development, sensor integration, communication systems, and software that enables autonomous robot operation during competitions.

Benjamin Villagrán
I am 18 years old and started studying robotics at Instituto de Innovación y Tecnología Aplicada when I was 15. I initially worked on the robot's electronics, learning about sensors, power systems, and control boards. As I gained experience, I expanded into software development, working with Raspberry Pi, Python, computer vision, serial communication, and artificial intelligence for victim detection.

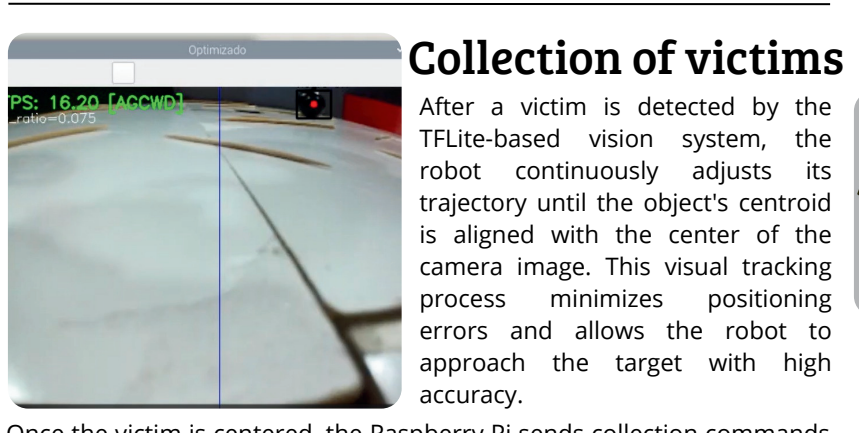
Today, I contribute to both hardware and software. My main responsibilities include AI and computer vision development, Raspberry Pi programming, electronics integration, 3D design with Fusion 360, and system documentation.

HARDWARE



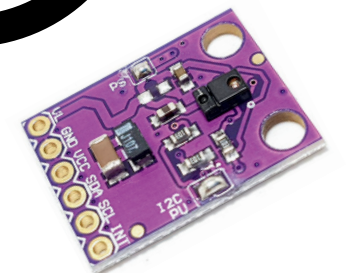
Our robot features a fully custom chassis designed by our team, composed of 3D-printed parts and a custom PCB that allows us to organize electronic components efficiently and simplify maintenance. It measures 157 mm long by 177 mm wide by 176 mm high and weighs 1,404 grams. For locomotion, we use front traction wheels that provide grip on the field and rear omnidirectional wheels that improve maneuverability during turns. The drive motors are equipped with encoders, allowing us to accurately measure the distance traveled and perform more precise movements. The robot's electronic architecture is based on a Teensy 4.1 and a Raspberry Pi 4. The Teensy handles real-time motor and sensor control, while the Raspberry Pi performs more advanced processing tasks.

Innovative Solution: 5-Servo Rescue Module



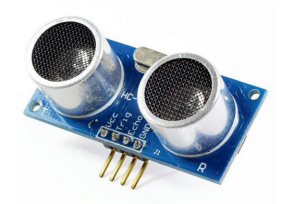
After a victim is detected by the TFLite-based vision system, the robot continuously adjusts its trajectory until the object's centroid is aligned with the center of the camera image. This visual tracking process minimizes positioning errors and allows the robot to approach the target with high accuracy. Once the victim is centered, the Raspberry Pi sends collection commands to the Teensy 4.1 through a custom UART protocol. The Teensy then executes the complete pickup sequence, coordinating the servo-driven gripper and lift mechanism to securely capture the victim and transfer it to the storage system.

APDS-9960:
Color detection and verification system. The APDS-9960 sensor is used to identify different colors present on the competition field and inside the rescue area.

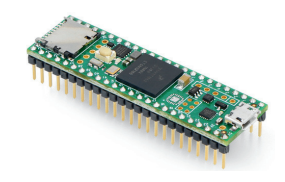


It provides an additional layer of verification for the camera-based detection system, improving reliability and reducing the possibility of false identifications during autonomous operation.

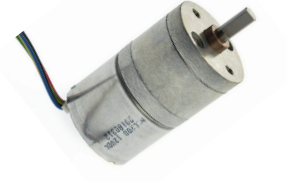
HC-SR04 (x3):
Distance measurement and obstacle detection. These ultrasonic sensors continuously monitor nearby walls and obstacles, providing environmental awareness that helps the robot navigate safely, avoid collisions, and make autonomous decisions during line following and rescue tasks.



Teensy 4.1:
Main robot controller. It processes sensor data, executes control algorithms, manages communication with the Raspberry Pi, and controls motors, servos, and other electronic components, ensuring reliable real-time operation.



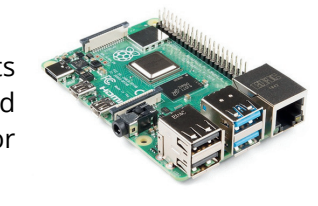
Brushless Motors DC (x4):
Robot locomotion system. These motors provide the power, speed, and precision required for movement. Their efficiency and reliability allow smooth navigation and accurate maneuvering throughout the competition.



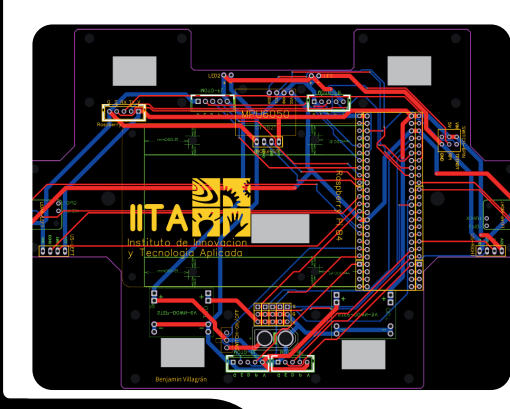
Servo motor 300°(x5):
Victim manipulation and sorting. The servos operate the claw, lifting mechanism, sorting system, and deposition structure, enabling precise victim collection, classification, and release during rescue operations.



Raspberry Pi 4:
Computer vision and image processing. The Raspberry Pi detects lines, intersections, green markers, rescue victims, and other field elements, generating navigation data that is sent to the Teensy for autonomous decision-making.



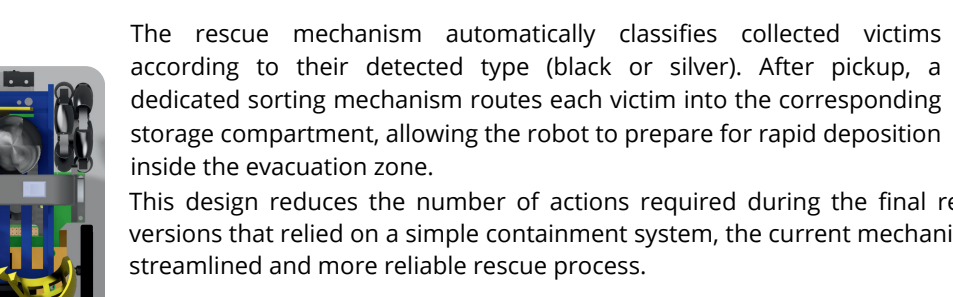
Innovative Solution: Custom PCB Design



One of the main electronic innovations of our robot is a custom-designed PCB that serves as the central hub for power distribution, signal routing, and subsystem integration. The board connects all major components, including the Teensy 4.1, Raspberry Pi, motor drivers, servos, BNO055 IMU, APDS9960 color sensor, VL53L0X distance sensors, ultrasonic sensors, indicators, relay, and control switches. To maximize system stability, the electronics were designed around separated power domains.

High-current actuators are powered through a dedicated 6.1 V rail, while the computing and sensing systems operate on an independent regulated 5.0 V rail.

This reduces noise, prevents voltage fluctuations, and improves overall system robustness during demanding rescue tasks. The PCB significantly reduces wiring complexity, improves serviceability, and allows rapid diagnostics and repairs during competitions. Compared to conventional wiring solutions, this approach provides a more compact, reliable, and scalable electronics architecture that supports the robot's advanced sensing, control, and artificial intelligence systems.



Teensy 4.1

The Teensy 4.1 is responsible for all real-time robot control tasks. Developed in C++ using PlatformIO, the firmware manages line following, sensor acquisition, motor control, and rescue-zone operations.

Raspberry Pi 4

The control system continuously processes information from the BNO055 IMU, VL53L0X time-of-flight sensors, ultrasonic sensors, color sensors, wheel encoders, and limit switches. This data is used to maintain stable navigation, detect obstacles and walls, identify ramps, estimate traveled distance, and execute precise maneuvers throughout the competition field.

To improve software organization, custom libraries were developed for both the drive system and the victim collection mechanism. This modular architecture simplifies testing, debugging, and future development while allowing different subsystems to operate independently.



The motor control system uses encoder feedback and PID algorithms to regulate speed and improve movement accuracy, enabling more consistent navigation during line following and rescue tasks.

Once a target is detected, the Raspberry Pi calculates its position relative to the camera frame. During victim collection, the robot continuously adjusts its movement until the detected victim is aligned with the center of the image, increasing collection accuracy and reducing positioning errors. The Raspberry Pi then sends navigation and rescue commands to the Teensy 4.1 through serial communication.

Communication System

Both processing units communicate through a custom UART protocol operating at 115200 baud. This architecture divides the computational workload between the two boards: the Raspberry Pi performs intensive image-processing tasks, while the Teensy handles low-latency control operations.

green_state	action	Robot Action
1	6	Turn Left Rotate to the left.
2	5	Turn Right Rotate to the right.
3	14	Double Green Maneuver Perform a double green maneuver (180° turn or according to the implemented rule).

The Raspberry Pi transmits steering commands, speed references, victim-detection results, and rescue-state information. The Teensy interprets these commands, executes the corresponding actions, monitors sensors, and controls actuators. In return, it sends status updates that allow the Raspberry Pi to coordinate transitions between stages such as line following, victim collection, and evacuation.

RESCUE AND EVACUATION ZONE

